

Document Number: P2605R0

Date: 2022-06-15

Audience: SG16

Reply-to: Tom Honermann <tom@honermann.net>

SG16: Unicode meeting summaries 2022-01-12 through 2022-06-08

Summaries of SG16 meetings are maintained at <https://github.com/sg16-unicode/sg16-meetings>.

This paper contains a snapshot of select meeting summaries from that repository.

- [January 12th, 2022](#)
- [January 26th, 2022](#)
- [February 9th, 2022](#)
- [February 23rd, 2022](#)
- [March 9th, 2022](#)
- [April 13th, 2022](#)
- [April 27th, 2022](#)
- [May 11th, 2022](#)
- [May 25th, 2022](#)
- [June 8th, 2022](#)

Previously published SG16 meeting summary papers:

- [P1080: SG16: Unicode meeting summaries 2018/03/28 - 2018/04/25](#)
- [P1137: SG16: Unicode meeting summaries 2018/05/16 - 2018/06/20](#)
- [P1237: SG16: Unicode meeting summaries 2018/07/11 - 2018/10/03](#)
- [P1422: SG16: Unicode meeting summaries 2018/10/17 - 2019/01/09](#)
- [P1666: SG16: Unicode meeting summaries 2019/01/23 - 2019/05/22](#)
- [P1896: SG16: Unicode meeting summaries 2019/06/12 - 2019/09/25](#)
- [P2009: SG16: Unicode meeting summaries 2019-10-09 through 2019-12-11](#)
- [P2179: SG16: Unicode meeting summaries 2020-01-08 through 2020-05-27](#)
- [P2217: SG16: Unicode meeting summaries 2020-06-10 through 2020-08-26](#)
- [P2253: SG16: Unicode meeting summaries 2020-09-09 through 2020-11-11](#)
- [P2352: SG16: Unicode meeting summaries 2020-12-09 through 2021-03-24](#)
- [P2397: SG16: Unicode meeting summaries 2021-04-14 through 2021-05-26](#)
- [P2512: SG16: Unicode meeting summaries 2021-06-09 through 2021-12-15](#)

January 12th, 2022

Draft agenda:

- [P2491R0: Text encodings follow-up](#)
 - Initial review.
- [P2498R0: Forward compatibility of text_encoding with additional encoding registries](#)
 - Initial review.

Attendees:

- Hubert Tong
- Jens Maurer
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- *[Editor's note: We did not have quorum due to low attendance; as a result, no polls were taken.]*
- *[Editor's note: Changes made in a draft of [P1885R9](#) were material to review of the papers on the agenda, so we first reviewed it.]*
- [P1885R9: Naming Text Encodings to Demystify Them](#)
 - Jens stated that there is a procedural question related to the changes made in this revision; the new draft changes the design after electronic polling.
 - Tom replied that the changes were at least partially inspired by comments received on the proposal by electronic polling participants and were intended to increase consensus.
 - PBrett noted that the polled revision did not have the wording to mandate `CHAR_BIT == 8` due to a procedural error and asked whether that warranted follow up with LEWG.
 - PBrett asked what it means in practice for the wording to mandate `CHAR_BIT == 8`.
 - Jens replied that it requires a call to a function specified with that wording to be ill-formed if the requirement is violated.
 - Jens explained that implementors can conform with this requirement by defining such functions as deleted, implementing them as a function template with an appropriate `static_assert`, or similar.
 - PBrett asked for more details regarding implementation as a function template and whether the functions could simply not be declared at all.
 - Tom replied that a function template is ok because taking the address of a standard library function is not allowed; the standard just requires a call expression to be well-formed.
 - Hubert explained that the name must be declared due to interaction with name lookup.
 - Jens stated an implementation preference to define the function as deleted.
 - PBrett: noticed that the revision history simply stated "Wording fixes" with no details.
 - Tom volunteered to review the R8 and R9 revisions and to summarize the differences in the meeting summary.
 - *[Editor's note: Tom did so; the wording differences identified include:*
 - *An additional `using enum id` declaration was added to the definition of `text_encoding`.*

- *The `wide_literal()`, `wide_environment()`, and `wide_environment_is()` declarations were removed and wording that referred to them adjusted accordingly.*
- *"character encoding" was changed to "character encoding scheme" in the paragraph following the note regarding how the names of `text_encoding::id` enumerators were derived from the IANA registry.*
- *A paragraph describing invariants maintained by `text_encoding` was added.*
- *Wording that introduced and used `SUBSTITUTE_UTF_ENCODING()` for mapping the bigendian and littleendian UTF-16 and UTF-32 schemes to their respective encoding forms was removed; related wording was adjusted accordingly.*
- *Wording to mandate `CHAR_BIT == 8` was added throughout.*

]

- [P2491R0: Text encodings follow-up](#)

- Jens summarized the paper.
- Jens stated that the changes in P1885R9 improve consistency with the IANA intent and P1885 usage.
- Jens added that there is opportunity for specification improvements, but that they can be addressed during LWG review.
- Tom noted that the wording plan described in the paper indicated intent to remove the requirement that `CHAR_BIT == 8`.
- Jens confirmed, but expressed a lack of concern; that restriction can be removed if motivation to do so arises.
- PBrett expressed a preference for explicit specification for how IANA octets are mapped to C++ code units.
- Jens agreed, stated that can be done during LWG review, and expressed a preference for a normative sentence that maps an IANA octet to a `char` code unit.
- Jens expressed a belief that relying on IANA for anything other than octets would be a mistake.
- PBrett asked if Jens had an alternative to suggest.
- Jens replied that he did not, but that such concerns usually arose during discussion of UTF-16 and UTF-32.
- Hubert stated that mapping to IANA in those cases is feasible, but noted that there are multiple possible mappings that may be platform dependent; particularly when the size of `char` is not 8 bits.
- PBrett agreed, but noted that a particular mapping could be required for a conforming implementation.
- Tom added that a non-conforming implementation would map to "other" in that case.

- [P2498R0: Forward compatibility of text_encoding with additional encoding registries](#)

- PBrett introduced the proposal:
 - Character encoding repositories other than IANA exist.
 - The mapping to IANA should be explicit such that a mapping to another registry could be gracefully introduced in the future if motivation arises.
- PBrett summarized the proposed changes:
 - Rename "id" to "iana_id"
 - Rename "mib" to "iana_mib"
 - Add recommended practice to avoid implementations creating an over dependence on IANA.

- PBrett asked for opinions on the proposed renames.
- Victor asked if other viable candidate registries exist in practice and stated that, if not, the proposed renames seem premature.
- PBrett replied that there are NB concerns about IANA being an unregulated, unaccountable, and unreliable organization.
- PBrett added that examples of other registries can be found in the [WhatWG Encoding Standard](#), IBM's [Character Data Representation Architecture \(CDRA\)](#), and [ISO/IEC 2022:1994](#).
- PBrett noted that the paper does not propose the addition of another registry; just the possibility to add more in the future in a seamless manner.
- Jens suggested adding examples of other registries to the paper.
- PBrett responded with concern that doing so might create disruption for the advancement of P1885 and result in considerable time spent debating whether the merits of each such repository warrant their being mentioned.
- Steve asked for confirmation that, assuming an additional registry, that a single text encoding ID is still needed.
- PBrett responded that the IANA ID is an enum class and that, in principle, multiple such classes are possible.
- Steve stated that renaming `mib()` to `iana_mib()` results in the feature no longer being generic.
- Jens agreed.
- PBrett responded that code written for P1885 today that consults `mib()` is necessarily concerned with IANA specifically.
- Steve asked what function a generic library should call then.
- Jens replied that, if the IANA ID is desired, then call `mib()`.
- Tom noted that, since multiple encodings may map to IANA's "other", reliance on `mib()` to uniquely identify an encoding is not possible.
- Hubert opined that the proposed renames are fine, but that extension to other registries might require different terminology.
- Jens offered examples such as "unique ID" and "UUID".
- Jens opined that it doesn't hurt to add "iana" to make the terminology association explicit.
- Hubert agreed.
- Hubert stated that, for wide encodings, there are some registries that are somewhat suitable; in CDRA, wide encodings aren't explicitly represented as they constitute a composition of a character set and an encoding.
- PBrett acknowledged and noted that the proposal is intended to clear design space for extension for such cases.
- Tom provided some arguments in favor of support for multiple registries:
 - As previously noted, the IANA specification is governed by an organization that some have concerns about.
 - The IANA registry is poor from a quality of specification perspective.
 - The IANA registry is missing entries that are found in other registries.
 - The same name is sometimes used by different registries to refer to different encodings.
 - Other registries are arguably more suitable for some environments; e.g., CDRA for IBM environments.

- Tom suggested that the proposal replace the P1885 proposed exposition only data members with post conditions on the default constructor to require `iana_mib()` and `name()` to return `id::other` and an empty string respectively; this is to avoid encouraging implementations to simply store an IANA ID.
- PBrett noted that much of the current wording is in terms of the `mib_exposition` only data member.
- Tom replied that those can be changed to `mib()`.
- Jens pointed out that the following text from the proposed wording creates the impression that the specified feature provides a remote API interface.
 - "[text.encoding] describes an interface for accessing the IANA Character Sets registry".
- Jens stated that `text_encoding` currently contains a list of all encodings in the IANA registry and that this proposal makes the `text_encoding` class more of a first class entity for which mapping to other registries is ancillary functionality.
- Jens opined that this direction suggests the need to create our own encoding registry since the functionality effectively defines one.
- Jens stated that such a change constitutes a change in direction that is more significant than the proposed renames suggest.
- PBrett acknowledged that the proposal makes the design more abstract in a manner similar to how Unicode specifies abstract characters.
- PBrett added that he could imagine a C++ appendix that lists the encodings, but that would then necessitate defining them.
- Jens stated that a registration service could be established, but did not advise doing so.
- Jens observed that discussion has not yet reached the bottom of how encodings would be mapped between encodings registered with different registries.
- Jens suggested the possibility of defining an `iana_text_encoding` class and later adding an `iso_text_encoding` class or similar for other registries if warranted.
- Hubert observed that the P1885 design contains the same mapping problem since it presents a single domain, but doesn't adhere solely to that domain.
- Hubert suggested the possibility of a `text_encoding` class template parameterized by a registry identifier.
- Tom noted that, if one were to map the encodings present in the [ICU converter explorer](#), then parameterization by registry is necessary due to conflicting use of the same name for different encodings.
- *[Editor's note: For example, "korean" maps to "windows-949-2000" via the "Windows" provider, but to "ibm-1363_P11B-1998" via the "IANA" provider.]*
- PBrett reiterated the intent of this proposal; to remove complete dependency on IANA.
- Tom stated that the proposed change is consistent with the P1885 direction given that comparison is dependent on name when an encoding maps to IANA's "other".
- Tom opined that there is no need to specify a mapping between repositories in the standard; the mapping can be implementation-defined.
- Jens agreed that leaving the mapping implementation-defined is possible but felt an obligation to specify the mapping.
- Hubert noted that, for implementors, the concern is what is in the interest of their users.
- PBrett expressed a belief that creation of a character encoding registry service would be outside the scope of WG21 work, but that he would be willing to assist with such an effort

outside of WG21.

- Jens agreed and noted that such a service would essentially be attending to graves.
- Hubert suggested that `text_encoding` may be more appropriate as a concept.
- Tom stated that distinct classes or class template specializations for each registry would create friction at interface boundaries.
- Hubert responded that a type erased interface could still be specified.
- PBrett expressed being open to other suggestions.
- Jens suggested renaming the current `text_encoding` class to `iana_text_encoding` and, if motivation arises for another registry, then a new class can be added.
- General discussion ensued regarding the ramification of distinct classes.
- Tom pondered what name would be returned by `name()` if multiple registries are supported.
- PBrett responded that his proposal intended to avoid that question.
- Tom stated that the next telecon will be held on 2022-01-26 and that the agenda will include:
 - [P2286R5 \(formatting ranges\)](#),
 - the LWG issues concerning `std::format` fill characters, and/or
 - a new revision of [P2498 \(Forward compatibility of text_encoding with additional encoding registries\)](#).

January 26th, 2022

Draft agenda:

- [P2286R6: Formatting Ranges](#)
 - Review proposed wording for new SG16 concerns and consistency with prior guidance.

Attendees:

- Charlie Barto
- Hubert Tong
- Jens Maurer
- Mark de Wever
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- Tom informed the group of tentative plans for a SSRG and SG16 joint telecon to discuss the security aspects of [P2528R0 \(C++ Identifier Security using Unicode Standard Annex 39\)](#) and asked for feedback or concerns about such a meeting.

- [P2286R6: Formatting Ranges:](#)

- Tom informed the group that Barry was unable to be in attendance but that we are ok to discuss the wording and gather feedback for him.
- Victor explained that he had assisted with the wording related to escape handling, but that Barry authored the rest.
- Jens stated that it is generally not advised to discuss a paper without the author present.
- Tom acknowledged the guidance and reported that he had communicated with Barry and that Barry was content with Victor being present to discuss any issues that are raised.
- Jens asked for confirmation that LEWG has already approved the design.
- Victor responded that the paper is present in the currently active electronic polling cycle.
- Victor shared the paper and reviewed the revision history.
- Victor began reviewing the wording.
- PBrett asked if the `formattable` concept has semantic constraints that cannot be expressed in the concept definition.
- Victor replied that it does not.
- Jens noted that, in section 5.1 of the paper, `[format.formattable]` paragraph 2 states the semantic requirements.
- Zach asked if the concept has been implemented as specified.
- Victor expressed uncertainty regarding the concept definition, but assured the group that the rest of the design has been implemented.
- Mark asked, with regard to section 5.2, why '?' appears as a type.
- Victor explained a requirement for mutual exclusivity.
- PBrett asked if there is intent to standardize the use of '?' to enable a debug representation generally; e.g., for non-standard types.
- Victor replied that doing so is outside the scope of the standard, but expressed support for that direction.
- PBrett noted that the paper introduces a `set_debug_format()` member function and asked if it would be desirable to add generic support for invoking it.
- Jens replied that the parser for the formatted type would presumably have to be responsible for recognizing the '?' character and invoking the member function.
- Jens noted that calls to `set_debug_format()` must activate the debug format regardless of whether a '?' is present in the format string.
- Jens suggested that adding generic facilities to support the '?' specifier would be ok, but probably best addressed by a different paper.
- Hubert noted that, in the proposed wording for `[format.string.escaped]`, subparagraph 2.4.1, that the "Other" ("C") value of `General_Category` is an abbreviation for a set of categories and requested that the wording specify the individual categories.
- *[Editor's note: The values for `General_Category` are specified in [table 12 of section 5.7.1 of Unicode 14 UAX#44](#).]*
- *[Editor's note: The wording also refers to the `General_Category` value of "Separator" ("Z") that is likewise an abbreviation for a set of categories and should presumably be expanded as well.]*
- Hubert noted that format stability cannot be guaranteed and that output will change when newer Unicode standards are adopted.
- Tom asked for confirmation that stability will only be lacking for currently unassigned characters.

- Steve replied that this should be further researched and noted that the "Unassigned" ("Cn") property is not stable.
- Hubert noted that the wording does not address non-Unicode text and asked if `isprint()` and `iswprint()` should be used to identify non-printable characters in those cases.
- Jens replied that doing so warrants further discussion.
- Hubert pondered whether it would be preferable to map non-Unicode characters to Unicode and then proceed with using the Unicode character properties.
- Tom noted that, for implementations that support user defined encodings, the implementation may not know how to map to Unicode.
- Hubert noted that such user defined definitions must define categories to be used for `isprint()` and other character classification functions, but acknowledged that a mapping to Unicode may not be present.
- PBrett stated that [format.string.escaped] paragraph 2 does not state how to determine if the string to be escaped is in a Unicode encoding.
- PBrett noted that he believes such wording to be present in the wording related to field width and suggested it may be desirable to generalize that and move it to a central location.
- Steve asked if the determination might be locale dependent.
- PBrett noted that previous guidance was that `std::format()` may be used for binary data and that any associated encoding is therefore tenuous.
- Jens suggested that, in those cases, a programmer might be advised not to use the '?' formatting type.
- Tom suggested it may be reasonable to, again, use the literal encoding as a proxy for the potentially locale dependent encoding.
- Victor agreed.
- Hubert stated that, for the non-Unicode case, determining printability would require either locale dependence or preservation of the compile-time literal encoding.
- Charlie noted that, historically, the latter would have been difficult and that, for Microsoft, the active code page (ACP) was used in the past.
- Hubert noted that, in a cross-compilation scenario, it is possible that the literal encoding is not a defined encoding for the target environment.
- Tom suggested that, at some point, we will need to poll whether the non-Unicode behavior should be locale dependent or not.
- Hubert expressed acceptance of non-locale dependent behavior for the non-Unicode case so long as there is no requirement to match the behavior for the Unicode case with regard to emitting hex vs UCN notation.
- Hubert noted that, if locale dependence is avoided, it will be necessary to assume an encoding for characters that are not consistently encoded for all locales; like '\ in EBCDIC environments.
- Hubert added that doing so might be ok if the choice is determined by the literal encoding.
- PBrett suggested it may be useful to support opt-in to locale dependence via the 'L' modifier.
- Victor stated that the 'L' modifier could be reserved for now.
- Mark noted that the 'L' modifier is currently supported for character type.
- Jens pointed out that the changes to [format.formatter.spec] paragraph 2 appear to indicate that a declaration of the `set_debug_format()` member function in any one of the

specializations will affect all of them.

- Victor agreed that this wording requires more work.
- Mark asked when a formatter object for which `set_debug_format()` was called is reset or what happens when the '?' specifier is not applicable to the type.
- Victor replied that it should be an error to specify mutually exclusive options or that the last option overrides prior ones but that further consideration is required.
- Jens stated that the order of the interior bullets in [format.string.escaped] subparagraph 2.2 need to be revised to address two issues:
 1. The algorithm must process the contents of string *s* in order.
 2. *s* consists of a sequence of code units, not UCS scalar values.
- Jens suggested trying to factor out the code unit and UCS scalar value cases to avoid the exceptions.
- Charlie stated that the handling of invalid code unit sequences needs to be specified since recovery may not always be possible; failure to recover could result in output containing a long sequence of values in hex notation.
- Jens acknowledged the scenario and agreed that the specification must be made clear about that.
- Jens asked what "universal character name escape sequence" is intended to mean in [format.string.escaped] subparagraph 2.4.2 and noted that a definition does not exist for "universal character name" though one does exist for *universal-character-name*.
- Jens noted that this wording probably should not refer to *universal-character-name* since it describes a grammar nonterminal and suggested replacing "its universal character name escape sequence" with "a sequence of scalar values".
- PBrett agreed.
- Steve pointed out that similar uses of grammar nonterminals appear in the wording.
- Jens agreed and suggested that the various "escape sequence" uses should be replaced with "a sequence of scalar values in the form ...".
- Jens pointed out a grammar issue in the first line of [format.string.escaped] paragraph 3; "... is equivalent to the escaped string representation a string of *c* ...".
- Hubert requested that a note be added to [format.string.escaped] paragraph 4 to indicate that behavior is not locale dependent but that the encoding may be informed by the literal encoding.
- Jens reported that the expected output noted in the comment for example *s*₃ in [format.string.escaped] paragraph 4 is incorrect; there should be two ranges and therefore two sets of brackets.
- Tom observed that the examples that follow [format.string.escaped] paragraph 4 depict the expected Unicode behavior, but appear to be part of paragraph 4 which is specific to non-Unicode behavior.
- Victor agreed there is a presentation issue that needs to be addressed there.
- Jens advised moving paragraphs 2 through 6 of [format.range] after the `range_formatter` class definition.
- Victor noted that "exposition only" should appear in italics.
- Mark asked why, in the last row of the table associated with [format.range] paragraph 6, "?"*s* is required as opposed to just "?".
- PBrett stated that requiring "?"*s* is inconsistent with the string case where "?" can be specified by itself.

- Victor explained that "s" indicates that a range is intended to be formatted as a string; "?s" is therefore needed to format the range as a string in debug mode.
- PBrett pointed out the "this is equivalent to invoking `set_brackets({}, {})`" note in [format.range] paragraph 5 and asked if a programmer is permitted to make such a call.
- Victor replied that the wording is intended for implementors.
- PBrett asked if, when implementing a range formatter, whether it is required to implement member functions like `set_brackets()`.
- Victor replied that LEWG had discussed this and that the member functions are intended to help avoid ABI issues.
- Mark asked if `set_separator()` and `set_brackets()` should be restricted to characters and how characters outside the basic character set need to be handled in various scenarios such as applicability to estimated field width determination.
- Victor replied that the functions accept strings as input.
- Tom reviewed the list of previously discussed design points that were noted in the [email announcing the agenda for the telecon](#).
- Tom noted the need for wording to address recovery from encoding errors.
- Charlie stated that recovery should follow the processes described in the [WHATWG Encoding Standard](#).
- [Editor's note: See [section 4.1, "Encoders and decoders"](#).]
- Tom noted the need to ensure that `std::filesystem::path` is rejected as a formattable range.
- Victor replied that it is rejected by the constraints on the `formatter` partial specialization specified in the [format.syn] updates; those constraints reject recursive ranges.
- Victor suggested it might be helpful to add a note that `std::filesystem::path` is rejected there.
- Tom announced that the next telecon will be February 9th.

February 9th, 2022

Draft agenda:

- [P2498R1: Forward compatibility of text_encoding with additional encoding registries](#)
 - Continue prior discussion and poll.
- [D2513R1: char8_t Compatibility and Portability Fixes](#)
 - Initial review.

Attendees:

- Charlie Barto
- Hubert Tong
- JeanHeyd Meneide
- Jens Maurer
- Peter Brett
- Steve Downey

- Tom Honermann
- Victor Zverovich

Meeting summary:

- [P2498R1: Forward compatibility of text_encoding with additional encoding registries](#)
 - PBrett presented:
 - The R0 revision was reviewed by SG16 a few weeks ago.
 - The R1 revision rebases the wording on the latest P1885 revision.
 - The wording was reworked to decouple IANA IDs from the exposition only data members.
 - Victor noticed an unnecessary trailing semicolon following the closing brace of the `std` namespace declaration in the proposed updates to `[text.encoding]`.
 - Hubert noted that Corentin's recently added response to P2498R1 in [P1885R10](#) noted unnecessary use of an enum for the proposed internal details of the `text_encoding` class and asked if it was necessary for it to be an enum.
 - PBrett responded that it is exposition only.
 - Jens pointed out an existing `using enum id` declaration in the `text_encoding` synopsis that should presumably have been changed to `using enum iana_id`.
 - Jens noted similar renaming updates needed in the postcondition for the `text_encoding(iana_id)` constructor where comparisons against `id::unknown` and `id::other` are currently present.
 - Jens observed that the `text_encoding(iana_id)` constructor does not state that its argument is stored.
 - PBrett explained that such storage is implied by the postcondition requirements on the result of calls to `iana_mib()`.
 - Jens suggested that there should be some wording that relates the exposition only `id` type to `iana_id`.
 - PBrett agreed that could be better specified.
 - PBrett indicated that some positive guidance is needed before spending further effort on this proposal.
 - Tom suggested polling support for the concerns the paper purports to address.
 - Discussion regarding polls ensued.
 - **Poll 1A: It should be more explicit in the identifiers used by the `text_encoding` interface that the facility is tied to the IANA character sets database.**
 - Attendance: 8
 - S F F N A S A
2 3 1 2 0
 - Weak consensus in favor.
 - **Poll 1B: The `text_encoding` class design should be modified to facilitate potential future association with additional encoding registries without retaining a bias towards the IANA registry.**
 - Attendance: 8 (one abstention)
 - S F F N A S A
1 3 2 0 1

- No consensus.
 - SA: I think IANA is a reasonable default and others can be added; we shouldn't slow down progress.
- Poll 1C: The `text_encoding` class should be renamed to `iana_text_encoding`.
 - Attendance: 8
 - SF F N A SA
 - 1 2 2 3 0
 - No consensus.
- Poll 1D: Address feedback on wording in P2498R1 "Forward compatibility of `text_encoding` with additional encoding registries", and forward the paper as revised to LEWG as a bug fix for P1885 "Naming text encodings to demystify them" with a recommended ship vehicle of C++23.
 - Attendance: 8 (one abstention)
 - SF F N A SA
 - 1 4 0 2 0
 - Weak consensus in favor.
- Jens requested that the changes regarding exposition only data members be removed if the intent is just to rename some identifiers as opposed to changing the original design intent.
- [D2513R1: char8_t Compatibility and Portability Fixes](#)
 - *[Editor's note: D2513R1 was the active paper under discussion at the telecon. The agenda and links used here reference P2513R1 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - JeanHeyd presented:
 - The `char8_t` related changes in C++20 made it more difficult to write code that works in both C and C++.
 - The proposal is intended to improve compatibility with C.
 - The proposal provides an escape hatch in C++ to restore some uses of UTF-8 string literals with `char`, `signed char`, and `unsigned char`.
 - The proposal only relaxes array initialization; pointer behavior is not changed.
 - The changes are intended as a DR against C++20 so that common code can be written for C++17, C++20, C++23, and C.
 - Victor stated that relaxing initialization for arrays but not for pointers creates an inconsistency and that he prefers an error.
 - Victor added that `signed char` and `unsigned char` are commonly used for `int8_t` and `uint8_t` and the proposed changes would allow arrays of these types to now be initialized by UTF-8 string literals.
 - JeanHeyd acknowledged this would be the case and is consistent with C++11 through C++17.
 - Hubert presented a backward compatibility concern involving overload resolution and initialization of parameters of aggregate type using list initialization syntax; the following example is well-formed in C++20, but would become ill-formed with the proposed change.


```
struct A { char8_t s[5]; };
struct B { char s[5]; };
void f(A);
```

```

void f(B);
void foo() {
    f({u8"text"}); // Resolves to f(A) in C++20, ambiguous with the proposed changes.
}

```

- PBrett observed that the overload resolution issue currently exists with signed char and unsigned char.
- Tom asked for confirmation that binding to a reference to an array would still be ill-formed.
- JeanHeyd replied affirmatively.
- Jens explained that there is an existing special case that permits initialization of arrays of signed char or unsigned char by an ordinary string literal.
- PBrett summarized; the overload scenario already exists for char, signed char, and unsigned char.
- Jens agreed and suggested adding an annex C entry with the above example to note the new ambiguity.
- JeanHeyd reviewed the wording.
 - The value of the `__cpp_char8_t` feature test macro is updated.
 - The substitution of "may" for "can" in `[dcl.init.string]` is a drive by fix done to maintain consistency with the other changes to the paragraph.
- JeanHeyd noted that, with the changes, a program that passes a UTF-8 string literal as an argument for a function parameter of type `const char*` remains ill-formed.
- JeanHeyd stated that the ability to initialize char-based arrays with a UTF-8 string literal addresses `constexpr` concerns in a much simpler way than is presented in [P1423R3 \(char8_t backward compatibility remediation\)](#).
- PBrett asked if any consideration was given for addressing these concerns with a core issue.
- JeanHeyd replied that a core issue wouldn't be appropriate since the C++20 changes were deliberate.
- Tom agreed and noted that this aspect of the C++20 changes was approved by EWG.
- Jens agreed that a core issue is unlikely to be helpful, especially since a paper already exists.
- Jens asserted that EWG should review and, as a DR, this can be considered after C++23 feature freeze.
- Hubert observed that the backward compatibility impact to overload resolution can be avoided by limiting the allowance to variables of array type.
- Jens agreed, but noted that would require more careful wordsmithing.
- **Poll 2: Add an Annex C entry and discussion to D2513R1, and forward the published paper as revised to EWG as a defect report.**
 - **Attendance: 8 (one abstention)**
 - **S F F N A S A**
1 5 0 1 0
 - **Strong consensus in favor.**
 - **A: Concerned about the different handling for pointers vs arrays; concerns increased as discussion continued.**
- Tom asked if anyone felt concern about the possibility of unintended initialization by a UTF-8 string literal as opposed to an ordinary string literal.

- Victor expressed minor concern, but that the risk is low.
- JeanHeyd noted that such unintended possibilities are already the case in C.
- Tom announced that the next telecon will be February 23rd.

February 23rd, 2022

Draft agenda:

- Discuss objectives and priorities for C++26

Attendees:

- Charlie Barto
- Hubert Tong
- Inbal Levi
- Jens Maurer
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- Tom reminded the group about the upcoming SSRG meeting on Monday, February 28th, and encouraged the contribution of suggested improvements for [P2528R0 \(C++ Identifier Security using Unicode Standard Annex 39\)](#) to the author.
- Discuss objectives and priorities for C++26:
 - Tom asked the group for suggestions on what we should focus on for C++26.
 - Tom suggested alias barriers as a remaining core language feature improvement.
 - [*Editor's note: See [SG16 issue #67](#).]*
 - PBrett suggested improvements for access to command line options and environment variables.
 - [*Editor's note: See [SG16 issue #66](#).]*
 - Jens noted the existing practice for the `_wmain()` entry point in Microsoft Visual C++.
 - PBrett noted the `envp` parameter that POSIX adds to `main()`.
 - Tom suggested `std::rope` for chained text segments.
 - Steve suggested transcoding and lamented JeanHeyd's absence.
 - PBrett suggested an owning string type that eschews null termination.
 - Charles expressed concern that a string type that doesn't ensure null termination may not improve security due to programmers attempting to pass such data to functions that require a null terminated string anyway.
 - PBrett suggested that implicit conversions could be prohibited.
 - Steve noted the usability challenges such prohibition creates.

- Victor stated that programmers will misuse such types even if implicit conversions are not supported; they will assume that, for example, a `data()` member function will provide a null terminated string.
- PBrett replied that such concerns apply to the existing `std::string_view` type as well.
- PBrett asserted there is room for a `std::string_view` like type that owns the string data.
- Charles noted that having to pass a string held in such a type to a function that requires null termination would require having to copy the string contents just to add a null terminator and lamented the cost of such copies.
- Jens asked PBrett what other features he might like to see in a new string type.
- PBrett responded that he would mostly like to not have to write yet more string types that avoid null termination concerns.
- Jens suggested that a string type that helps to avoid allocation and deallocation costs could be beneficial.
- Jens stated he would prefer not to spend time in SG16 on general data structures.
- Jens asserted that the elephant in the room is ICU and the functionality it provides and expressed a desire for a roadmap towards providing similar functionality.
- Steve suggested that PBrett publish a paper extolling the benefits of a string type without null termination.
- PBrett responded that benefits appear when working with databases and graphics.
- Tom stated that, if a new string type were to materialize, he would like to know for sure what encoding it is intended to be used with.
- PBrett offered two categories of such encoding assurances: 1) a type intended for text with an assumed encoding, and 2) a type that enforces well-formed encoding.
- Steve offered a third category: 3) a type that maintains invariants like normalization form.
- Steve reported having to investigate problems due to assumption of UTF-8 leading to errors when data was passed to Python programs that enforced well-formedness.
- Steve stated that Bloomberg is strongly in favor of a type that maintains such invariants.
- Charles expressed desire for a way to pass a filename on the command line that is preserved.
- Charles explained that this currently isn't possible on Windows due to transcoding that occurs when preparing a command line to pass to `main()`.
- Tom asked if it is possible to enter such names on the command line from Windows shells in the first place, but acknowledged it can be done from `CreateProcess()`.
- Charles replied that tab completion and similar shell features can produce such names.
- Tom asked to confirm that this should be considered a language issue as opposed to a shell issue.
- Charles explained that other languages provide such mechanisms and noted that Rust has an `osString` type that stores WTF-8 on Windows.
- PBrett added that Rust is a good example of a language that provides strings that don't guarantee null termination.
- Tom returned to discussion about features to focus on and suggested Unicode algorithms.
- Hubert asked if ICU provides message formatting features.
- Tom confirmed that it does.
- PBrett noted the work that the Message Format Working Group (MFWG) is doing.
- [Editor's note: The MFWG tracks its work [here](#).]
- Tom suggested improved support for regular expressions as another feature to work on.

- Charles replied that such work depends on whether the committee is willing to standardize a `std::regex2` or similar.
- Charles stated that a new regex design would presumably face the same challenges that `std::regex` did and may therefore produce a similarly poor result.
- Charles noted that regular expression support is a feature for which interoperability across libraries is not often needed.
- PBrett asserted that a standard regex facility is beneficial because general programmers aren't particularly good at writing their own.
- Charles suggested that a class intended to be used as a base class for string buffer management could be beneficial for building parsers; particularly a type that provides interfaces to peek ahead and push back.
- PBrett suggested a `std::lexy` with reference to Jonathan Müller's work.
- *[Editor's note: Jonathan's Lexy work is published [here](#).]*
- Tom asked if the work done on `std::format` provides a precedent for how to design a `std::regex` replacement that would be more isolated from ABI concerns.
- Charles responded that `std::format` is fairly exposed to ABI concerns, but acknowledged the possibility of designing a more isolated type.
- Victor stated that Hana Dusíková's CTRE provides a good example of how to write parsers.
- *[Editor's note: Hana's CTRE work is published [here](#).]*
- PBrett asked about ideas for how to be more explicit about associating an encoding with existing text; e.g., how to have text in UTF-8 and pass it to something that requires UTF-16.
- Steve asked if anyone recalled a proposal that would allow specifically binding to a string literal.
- Tom stated that he did not but noted that a user-defined literal (UDL) can provide similar functionality.
- *[Editor's note: Tom was thinking of something like this:*

```
#include <cstdint>
class string_literal {
private:
    const char *p;
    string_literal(const char *p) : p(p) {}
public:
    const char* data() const { return p; }
    friend string_literal operator ""sl(const char *p, std::size_t);
};
string_literal operator ""sl(const char *p, std::size_t) {
    return string_literal(p);
}
void f(const char*);
void g(string_literal sl) {
    f(sl.data());
}
void h() {
    g("hello"sl);
```

```
}  
]
```

- Steve stated that a facility that allows associating a dynamic encoding would be useful for scenarios in which the text and its associated encoding are not known at compile time.
- Victor noted that encodings are often properties of interfaces and suggested that an attribute based approach to specifying encoding expectations could be helpful; this would allow specifying expectations that are dependent on locale or Windows Active Code Page (ACP).
- Charles expressed support for that approach.
- PBrett asked what the advantage of an attribute would be.
- Victor replied that it could be added to existing functions without ABI impact.
- Charles observed that, if the `#embed` is adopted, then text that is not in the literal encoding might be imported.
- Hubert responded that the `#embed` author has been clear that the proposal is intended to provide binary resources.
- Tom changed topics by asking for suggestions regarding how to solicit new SG16 attendees and reported that he and Peter had previously discussed inviting other WG21 members to share what the organizations they work with do in practice.
- PBrett responded that backward compatibility and existing practice are motivation for not providing new facilities.
- PBrett suggested that, if someone were to come forward with a proposal to adopt a suite of string types similar to those provided by Rust, that we might tell them no thank you.
- PBrett stated that we should, of course, discuss any proposals that come before us.
- Tom replied that seems to argue for the status quo plus what people actually do.
- Jens expressed belief that there is good opportunity to make progress with small improvements and provided a UTF-8 decoding iterator as an example.
- Tom replied that an existing proposal covers such iterators.
- *[Editor's note: See [P0244 \(Text view: A C++ concepts and range based character encoding and code point enumeration library\)](#).]*
- Charles responded that he has written a grapheme iterator.
- Charles noted that such iterators are always going to be slower than bulk conversion routines.
- PBrett suggested adding locale-independent replacements for character classification functions like `isdigit()`; e.g., `is_ascii_digit()`.
- Charles expressed support for such locale invariant functions and noted via chat that `glib` provides such a variant named `g_ascii_isdigit()`.
- Tom noted that there is a distinction; locale-independent variants would operate using an implementation-defined encoding where as ASCII variants would operate using ASCII even in an EBCDIC environment.
- Hubert asked if anyone would object to having both variants; e.g., a locale-independent variant that always operates using the "C" locale and an ASCII variant.
- No objections were raised.
- PBrett stated that the highest priority item should be transcoding facilities.
- Jens asked if he meant something like `iconv()`.
- PBrett responded that he meant something more like JeanHeyd Meneide's `ztd::text`.
- *[Editor's note: JeanHeyd's `ztd::text` work is published [here](#).]*

- Charles stated that, with regard to providing an ICU interface in the standard, there are performance implications; support for generic iterators would warrant availability of definitions for inlining purposes.
- PBrett asked if anyone could comment regarding the utility of `std::message`.
- Steve responded that the main problem with message catalogs is that they don't tend to work well with real languages.
- PBrett reported having good experience with GNU gettext, but that it required discipline to be used successfully.
- PBrett stated that the other elephant in the room is locale, but noted that `std::format()` provides a foundation we can build on.
- Tom suggested the possibility of enabling thread-specific locale facilities that avoid reliance on a global locale.
- PBrett suggested a review of all locale dependent interfaces followed by a proposal that adds variants that accept a locale as an argument.
- Jens replied that C already does that and that all C++ interfaces use `std::locale`.
- Hubert stated that POSIX provides a thread aware locale interface that enables changing locale for a specific thread as well as interfaces that accept a locale argument.
- PBrett provided an example; a `std::isalpha()` overload that accepts a `std::locale` argument.
- Hubert responded that POSIX specifies a `isalpha_l()` interface that accepts a `locale_t` argument.
- Tom asked if `std::locale` is fundamentally problematic or whether the problems we face with it are isolated to the facets.
- PBrett responded that it has fundamental limitations due to reliance on inheritance.
- Victor stated that the facet's assumption that characters fit in a code unit is the most significant problem.
- Victor added that `std::locale` uses reference counting and therefore has a performance profile similar to a shared pointer.
- Hubert stated that `std::locale` strongly ties C++ implementations to the suite of C locales since the interface depends on the names of C locales; it is therefore questionable how implementors might provide more generic locale support.
- Steve stated that WG14 recently adopted a proposal that adds '@', '\$', and `` to the basic source character set.
- Steve added that he intends to bring a matching paper to WG21 in the near future.
- [*Editor's note: The adopted WG14 proposal is [N2701](#).*]
- Tom announced that the next meeting will be on March 9th and, unless a new paper materializes, will likely focus on diving deeper into one of the topics discussed today.

March 9th, 2022

Draft agenda:

- ICU features to consider for C++26

Attendees:

- Charlie Barto
- Hubert Tong
- Jens Maurer
- Peter Brett
- Steve Downey
- Tom Honermann

Meeting summary:

- Tom introduced the topic:
 - Tom was inspired by Jens' assertion during the previous telecon that "the elephant in the room is ICU and the functionality it provides".
 - Based on that, Tom created a [Google Doc](#) containing a list of ICU feature categories annotated with Tom's opinions on whether such features are worthy of consideration for standardization in C++26 should suitable proposals materialize.
 - Tom would like to issue a Call for Papers covering the features we agree are worth consideration.
- Tom began reviewing the feature categories in the order they appear in the document.
- The categories enumerated below are those for which there were material comments.
- "Unicode character properties":
 - Tom pondered the benefits of exposing the set of raw UCD properties as opposed to just those that are needed for other features.
 - Charles responded that there are representation choices to be made based on usage.
 - Steve noted that support for segmentation requires many of the UCD properties.
 - Steve stated that it would be useful to expose all of the properties to enable experimentation.
 - Steve noted that, if there are design problems with how the data is exposed, that would be useful information.
 - PBrett observed that the UCD properties are needed to implement proper internationalization and localization support.
 - PBrett pondered whether exposing all UCD properties might create portability issues.
 - Tom asked if it might be reasonable to only expose properties with stability guarantees.
 - Steve replied that doing so probably would not be viable; some properties required for segmentation are not stable and there is a need to be able to fix things.
 - Tom noted the difference between data being stable as compared to property shapes being stable.
 - Steve acknowledged and noted that property shapes haven't changed in a while.
 - Steve added that the ICU maintainers would presumably push back hard if Unicode made significant changes to the shape of existing properties.
- "Sets of Unicode Code Points and Strings":
 - Tom commented that the items in this category appear to be simple utilities.
 - PBrett opined that may be a good reason to provide them.
 - Tom observed that they appear to be used to provide support for character classes.

- "Locales":
 - PBrett stated that one of the challenges with locale identifiers is that choice of locale is not fixed at compile-time.
 - PBrett added that locale IDs are subject to geopolitical changes.
 - Steve reported that there has been much work on client-side localization through [ECMA 402](#), and [ICU4X](#).
 - Steve expressed caution over trying to standardize something like ECMA 401 and ICU4X given that they are recent developments.
 - Steve noted that client side support uses browser based facilities.
 - PBrett wondered if C++ code compiled to WASM should transparently proxy to the local environment.
 - Tom asked whether new locale support could be built on `std::locale`, perhaps via new facets.
 - Hubert objected to such support built on new `std::locale` facets due to `std::locale` being dependent on C locale names.
 - PBrett replied that it would be useful to be able to get an ISO locale name from a standard locale.
 - Hubert noted that encoding information would be lost in that case.
 - PBrett agreed and asserted that encoding ideally wouldn't be a locale concern anyway.
 - Hubert replied that a particular script could be implied by `std::locale`.
 - Steve observed that we seem to be in agreement that we don't know how to do this today.
 - Tom stated that the problem is that proper case mapping, case folding, and collation can't be provided without proper locale support.
 - PBrett agreed and asserted the need for a plan to improve locale support.
- "Resource Bundles":
 - Tom suggested that this feature could be built on top of features like `#embed` and support for dynamic libraries; perhaps a downstream feature then.
 - Charles noted that Microsoft style resource management can be provided on other platforms using linker scripts.
 - PBrett opined that resources are a requirement for a good locale system.
- "Calendars and Time Zones" and "Date and Time Formatting":
 - Tom stated that base facilities exist that could be expanded, but that there are locale dependencies.
 - PBrett acknowledged concerns raised by Corentin on the mailing list that noted that the existing facilities are not at parity with the ICU features.
- "Message Formatting":
 - Tom stated that this is awaiting further developments from the [Message Format Working Group \(MFWG\)](#).
 - PBrett commented that, what programmers want and what they need are not always the same thing.
 - Charles expressed a desire for experience outside the standard before pursuing support in the standard; even more so for this feature than for other ones.
 - Charles added that, ideally, a production quality application would be built on top of such a facility; one that has users using it from multiple locales.
 - PBrett opined that `std::format()` may not provide a great base to build this on.
 - Charles replied that it is probably usable, but not sufficient as is.

- Tom stated there would presumably be a need to pass a message ID in place of a format string.
- Charles reported that could be done using `std::vformat()`.
- "Text Transformation":
 - Tom suggested that, if there is a specific transliteration that we have motivation to provide, then we can do so; range adapters provide general support for this otherwise.
 - Steve noted that JeanHeyd's [ztd.text](#) library can do some of this.
 - Hubert asked if this feature category is referring to technical transliteration or linguistic transliteration.
 - Tom replied that he did not know.
- "Bidirectional Algorithm":
 - Tom asked for confirmation that this does not have locale dependencies and is effectively about state management for layout purposes.
 - Steve replied that the algorithm is complicated as it tries to support all possible scenarios.
- "Collation":
 - Jens described a design possibility involving locale independent base functionality that operates on a set of locale dependent collation rules.
 - Jens stated that the rules engine would have to be sufficient to perform all tasks required by all locales.
 - Jens noted that such a facility could be useful as a building block for locale support and as a standalone facility.
- "String Searching":
 - Tom noted the locale dependence.
 - PBrett noted collation dependence.
 - Jens stated that precise requirements are needed and that basic element searching is present.
 - Hubert suggested this might be provided by a regular expression facility.
- "Text Boundary Analysis":
 - Steve stated there are natural range interfaces for this and that the interface is probably clear.
 - Hubert asked when a C++ programmer would use this.
 - Steve replied that he performs such analysis for reflowing text in outgoing email.
 - PBrett added that such support was an integral part of a DSP language he worked on.
 - Steve stated that word wrapping comes up in many scenarios.
- "Regular Expressions":
 - Tom suggested we consider this category if a paper arrives.
 - Tom noted that a proposed design would be locale dependent and would presumably have to operate on various string types, potentially including segmented text data structures.
 - PBrett pondered the possibility of a run-time only library with a narrow interface that could later be overloaded for new extensions.
 - Steve stated that this is not an area of his expertise but that he would be willing to help author a paper if a good design were to be proposed.
 - Steve noted that there are many SG16 and LEWG related issues to consider.
- "StringPrep":

- Hubert stated that this feature is related to string normalization for interchange purposes such as key lookup.
- Jens noted that the [stringprep RFC](#) specifies exclusion of certain characters and expressed a desire to better understand what this facility is used for.
- Tom updated the Google Doc to change the consideration column value from "N/A" to "No", to add a reference to RFC 3454, and to note that this is a string normalization feature.
- "IDNA":
 - Tom expressed considerable ignorance of this topic.
 - [*Editor's note: "IDNA" stands for "Internationalizing Domain Names for Applications" and is covered by [UTS #46](#).]*
 - PBrett suggested such support might be needed for standard networking.
- "Universal Time Scale":
 - Tom suggested this is not an SG16 concern and could be tackled directly by LEWG if motivated.
 - Tom updated the Google Doc to change the consideration column value from "N/A" to "No; LEWG".
- "Paragraph Layout / Complex Text Layout":
 - Review of the ICU `pLayout.h` header revealed that this is an experimental facility.
 - Tom updated the Google Doc to change the consideration column value from "Not yet" to "No" and added a note about the facility being experimental.
- "ICU I/O":
 - Tom stated this is not an SG16 concern.
- Tom stated that the next telecon is scheduled for 3/23; the agenda is TBD.

April 13th, 2022

Draft agenda:

- [P2558R0: Add @, \\$, and ` to the basic character set](#)
 - Initial review.
- [D2572R0: std::format\(\) fill character allowances](#)
 - Initial review.

Attendees:

- Charles Barto
- Hubert Tong
- Jens Maurer
- Mark de Wever
- Peter Bindels
- Peter Brett
- Steve Downey
- Tom Honermann

- Victor Zverovich

Meeting summary:

- [P2558R0: Add @, \\$, and ` to the basic character set](#)
 - PBrett asked whether this proposal is an SG16 concern.
 - Tom replied that we are the encoding experts and best equipped within the committee to evaluate whether these changes will have consequences for source character sets used in practice; our affirmation of the proposal will hopefully ease progress through other groups.
 - Charlie asked for confirmation that SG22 will review the paper as well.
 - Steve replied affirmatively and stated WG14 has already adopted the proposal for C.
 - Someone (apologies, the editor neglected to record who) noted the existence of [P2342: For a Few Punctuators More](#) and that it argues that these characters could be used as new operators.
 - Tom responded that P2342 is an example of a paper that is not an SG16 concern; once the characters are available in the source character set, how they are used is an EWG concern.
 - Hubert observed that this introduces a requirement that these characters be encoded as a single code unit.
 - Jens acknowledged and noted that this is required by [\[lex.charset\]p6](#) for all characters in the basic literal character set.
 - Jens requested that the paper prose discuss this requirement.
 - Steve agreed to add such prose.
 - Jens stated that he does not know if this requirement would be problematic for any existing character sets.
 - Steve replied that there are infrequently used EBCDIC code pages that lack them.
 - Jens asked if those code pages also lack other characters.
 - Steve responded that they probably do.
 - Charlie asked if those problematic EBCDIC code pages have unused code points that could be used for these characters.
 - Tom suggested that digraphs could be introduced to support those code pages if needed.
 - Jens replied that digraphs can't be used inside character or string literals.
 - Steve suggested this concern can be addressed if these characters start getting used within the standard.
 - Jens reported that the addition of these characters to the basic character set makes them ineligible to be specified via a *universal-character-name* (UCN) outside of a character or string literal due to [\[lex.charset\]p3](#).
 - PBrett suggested that restriction could be lifted.
 - Charlie stated that existing uses of '\$' are probably limited to identifiers and symbol renaming via attributes, pragma directives, or `asm` labels.
 - Hubert reported they may appear in preprocessor stringization.
 - PBrett reported they may appear in header names as well.
 - Jens noticed that use of a UCN to `#include` a source file named with one of these characters would become ill-formed.

- PBrett reiterated support for lifting restrictions on use of UCNs to name characters from the basic character set with the rationale that we shouldn't ban things that are only bad ideas.
- Charlie stated that, if such incompatibilities were to cause problems in practice, then lifting that restriction would be the obvious solution.
- Steve suggested such concerns can be addressed after the paper is approved; we know programmers want to use these characters.
- Tom asked Steve if he knows whether the UCN concern was discussed in WG14.
- Steve replied that it was not.
- Hubert asked when UCNs are translated in identifiers and other preprocessor tokens.
- Jens replied, during translation phase 3 except in quoted contexts; so translation occurs as a *preprocessing-token* is formed.
- PBrett observed that the new UCN restrictions would be limited to *h-char* and *q-char* sequences since these characters aren't otherwise usable outside quoted contexts.
- Jens replied that they may also appear in a *preprocessing-token* used in stringization.
- Hubert stated that a UCN specifying one of these characters would become ill-formed during stringization.
- Hubert added that, likewise, use of a UCN to specify '\$' in an identifier would become ill-formed.
- Tom asked whether that matters since use of '\$' in an identifier is already an extension.
- Jens noted that these characters currently match the "each non-whitespace character that cannot be one of the above" case of [preprocessing-token](#).
- Jens requested that this discussion be included in the paper.
- PBrett asked whether *h-char* and *q-char* sequences remain a backward compatibility concern.
- Jens replied that they do, but that UCNs in such sequences already have implementation-defined behavior; what UCNs mean in *h-char* and *q-char* sequences isn't really defined.
- [Editor's note: Per [\[lex.phases\]p1.3](#), UCNs are not recognized and replaced in *h-char-sequence* and *q-char-sequence* sequences and, per [\[lex.header\]p1](#), these sequences are mapped in an implementation-defined manner.]
- Tom suggested that it might be worth updating the paper to describe how existing implementations behave with respect to UCNs in preprocessor token concatenation, stringization and `#include` scenarios.
- Jens asked if there are other ways in which these characters might plausibly be used today.
- Tom replied that Objective-C uses '@'.
- Jens mentioned the possibility of concerns being raised regarding the imposition of single code unit encoding for these characters on the execution character set.
- Tom asked Hubert if he was aware of any EBCDIC related concerns.
- Hubert replied that his colleagues in WG14 did not express such concerns and that he is confident that they would have if they had any.
- Hubert noted that locales that don't support these characters would no longer be strictly conforming but could still be supported as extensions.
- Tom summarized the discussion; there are requests to Steve to update the prose for several of the items discussed, but that there do not appear to be any objections to the paper direction.

- [D2572R0: std::format\(\) fill character allowances](#)
 - Tom provided an overview of the topic and prior discussions.
 - Tom asked for additional categories of characters that should be represented in the introductory table.
 - Tom stated that Zero-Width Joiner (ZWJ) and Zero-Width Non-Joiner (ZWNJ) cases were not added because he thought they were not interesting.
 - Tom noted that lone surrogates should not be possible.
 - PBrett suggested the bidirectional override characters.
 - *[Editor's note: the bidirectional override characters are U+2066 through U+202E.]*
 - Tom agreed to add right-to-left and left-to-right examples.
 - Tom stated that extending fill character support to arbitrary extended grapheme clusters (EGCs) in the future would presumably impose an ABI break.
 - Mark agreed that it would; at least one implementation only stores a single code unit as the current wording appears to specify.
 - Charlie agreed and noted that another implementation stores the fill character as a code unit sequence with a maximum length of 4 code units for UTF-8.
 - Tom noted that, for the "Estimated display width restrictions" section, there is no good or right solution.
 - Tom asked if characters with an estimated width other than one were to be banned, how that would be accomplished without imposing undesirable overhead.
 - PBindels noted that the ZWSP case is interesting; if it were given an estimated-width of 0, then an infinite amount of padding would be required.
 - Charlie stated that the estimated width is not intended to be accurate; it is best effort.
 - Charlie stated that, in the "Existing practice" section, `std::format_error` is now thrown for the cases that previously produced the stack overflow for MSVC.
 - Charlie noted that the diagnostic is somewhat disappointing though since an invalid type is reported because the intended fill character does not match the `fill-and-align` grammar.
 - *[Editor's note: The diagnostic produced by gcc with `{fmt}` is likewise disappointing.]*
 - PBrett suggested that a table that demonstrates the desired output inline with the proposal would be useful.
 - Victor stated that the paper direction makes sense and is consistent with previous guidance.
 - Victor pondered the consequences of diagnosing fill characters with an estimated width other than one; on one hand, it would be nice, but on the other hand, it adds overhead and potentially restricts valid use cases.
 - PBrett suggested that fill characters with an estimated width other than one could be conditionally supported.
 - PBrett stated that his preferred approach would be to diagnose at run-time (e.g., throw an exception) when alignment requirements could not be achieved due to the estimated width of the fill character.
 - PBindels reviewed the "Proposal" section and stated:
 - The first point to restrict to a single UCS scalar value seems sensible.
 - The third and fourth points appear to be consistent with what implementations currently do.
 - The second point is the questionable one.

- PBrett suggested that the number of fill characters inserted could be unspecified when the fill character has an estimated width other than one.
- PBindels replied that he considered that as well, but since the actual width is dependent on font selection, a consistent result may not be achieved anyway.
- Charlie reminded the group that the estimated widths are not currently specified by any standard.
- Charlie observed that diagnosing fill characters with an estimated width other than one will have the potential effect of rendering existing code invalid if estimated widths are changed in the future.
- PBrett suggested another possibility that, if the fill character has an estimated width of two, then perform the alignment as if the fill character and all characters in the format argument have an estimated width of one; this would enable idiographic characters to be formatted properly.
- Charlie responded that, if such a feature is desirable, then it would be preferable to add a flag to opt-in to it rather than inferring it based on the chosen fill character.
- Tom agreed and noted that idiographic characters are just one special case.
- Charlie stated that, for table style alignment, it is likely preferable to emit a field separator character explicitly in the format string rather than to rely on the fill and align capabilities.
- PBindels asserted that there is value to having well-defined portable behavior across implementations, so unspecified and implementation-defined behaviors should be avoided where possible.
- Hubert asked if Victor has good examples of this facility being used with format arguments that have characters with estimated widths other than one regardless of fill character.
- Victor replied affirmatively, but stated he did not recall specific examples; such cases involved terminal output.
- Tom reported vague recollections of such examples being present in Victor's original papers.
- PBrett stated that support for certain languages remains a concern for him and reported seeing Japanese characters reliably displayed in terminals in tabular formats.
- Tom asked if he knew how the programmers were facilitating such output.
- PBrett replied that he did not.
- PBindels asked if it would be feasible to research what features would be useful for such languages.
- PBrett replied that it isn't feasible for him to do so due to the number of possible solutions; he would like to allow implementations to be creative and see what the results bring.
- PBindels again emphasized a desire for portable behavior.
- Charlie agreed and stated that a creative solution in one implementation might produce an error in another.
- Tom asked Peter Brett if it would suffice for an implementation to provide a flag to opt-in to an alternate behavior.
- PBrett lamented the absence of attendees that are experts in non-Latin based languages.
- Hubert questioned the frequency with which a programmer would want to use a fill character with an estimated width other than one; the programmer would presumably need to be able to specify a fill-remainder character or otherwise provide their own padding.

- Victor agreed with the goal of specifying consistent behavior and suggested standardizing the demonstrated existing behavior in which a fill character is assumed to have an estimated width of one.
- Charlie noted that a programmer can implement their own fancy formatter that produces a result that is then embedded using standard formatters.
- Tom stated that it sounds like we have a few possible extension mechanisms that can be used for experimentation, work arounds, or future standard behavior.
- Tom reported that he is leaning towards Victor's suggestion of specifying the currently demonstrated implementation experience.
- PBindels indicated he would be content with any of the demonstrated outputs so long as behavior is consistent across implementations.
- PBrett expressed concern about specifying particular behavior in the absence of more diverse expertise in SG16.
- PBindels stated that he would reach out within his organization to try to find people with more diverse experience that would be interested in attending.
- Tom reported that the next meeting will be on 2022-04-27.

April 27th, 2022

Draft agenda:

- [P2286R7: Formatting Ranges](#)
 - Review recent updates and confirm direction.
- [P2558R1: Add @, \\$, and ` to the basic character set](#)
 - Review recent updates.

Attendees:

- Hubert Tong
- Jens Maurer
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich
- Zach Laine

Meeting summary:

- [P2286R7: Formatting Ranges](#)
 - *[Editor's note: D2286R7 was the active paper under discussion at the telecon. The agenda and links used here reference P2286R7 since the links to the draft paper were not shared publicly. The published document may differ from the reviewed draft revision.]*
 - PBrett provided an introduction.

- Victor explained that LWG reviewed the wording and conditionally approved the paper subject to SG16 review.
- PBrett recalled concerns raised in the past regarding the use of Unicode properties.
- Victor replied that implementors were present during the LWG review and did not express any objections or concerns.
- Victor noted that Hubert provided some wording tweaks during the LWG review.
- PBrett presented wording updates Tom [proposed on the SG16 mailing list](#).
- PBrett expressed a preference for Tom's wording relative to the current wording.
- Victor stated that he is ok with Tom's wording so long as it is equivalent to the current wording in the paper for the Unicode case.
- Victor stated that he would prefer not to defer to other format facilities for the description of how hexadecimal values are formatted.
- Hubert expressed a desire to document spacing and non-printable characters by their encoded values as opposed to their character names or glyphs.
- Hubert explained that doing so would free the library from having to be aware of the literal encoding selected at compile-time.
- Tom acknowledged the concern; the set of spacing and non-printable characters, or their encoded values may differ for one literal encoding vs another.
- Hubert noted that the concern applies to both EBCDIC and Windows code pages.
- Hubert stated that the proposed wording could produce strange results in cases where unnecessary shift states are present.
- PBrett observed that the current wording does not state which encoding is used in cases where printable characters overlap with control characters in a related encoding, but the proposed wording does.
- Jens noticed that the proposed wording states that the literal encoding is used to construct *E*, but not to interpret *S*.
- Tom acknowledged the omission and stated that it needs to be corrected.
- PBrett returned to his example where a locale encoding overlays graphical characters over control characters and noted that the overlaid characters would be interpreted in the literal encoding.
- Hubert reported that his implementations are not affected by overlay concerns and that locale support can be added later if motivated.
- Zach asked if the method for determining whether *S* is in a Unicode encoding matches the method specified for `std::format()` in C++20.
- Tom replied that he didn't recall how it was specified.
- *[Editor's note: It does not appear to be specified. The relevant wording simply states "For a string in a Unicode encoding, ...". See [\[format.string.std\]p11](#), [\[format.string.std\]p12](#), and [\[format.string.std\]p14](#). Improvements appear to be warranted.]*
- Jens stated that, with the exception of the use of *CE* for string *S*, this is well-specified so long as it matches the desired behavior.
- Tom noted that `std::format()` is intentionally locale independent.
- Hubert reported that his implementations will likely assume a certain literal encoding rather than storing the literal encoding actually used at compile-time; that encoding is likely to be EBCDIC 1047 or, for ASCII contexts, ISO-8859-1.
- Hubert expressed his understanding of the design intent to be that an escaped sequence can be interpreted to reproduce the original byte sequence.

- Hubert suggested that it may be worth adding a note to that effect.
- Tom acknowledged the intent and noted that his wording fails to reflect that intent for stateful encodings since state transitions in *S* should be reflected as escape sequences rather than interpreted when constructing *E*.
- PBrett asked for other concerns.
- Tom noted that there is the issue of handling boundaries of ill-formed code unit sequences and asked if anyone wanted to argue for addressing that now.
- PBrett expressed a preference not to address it now.
- Hubert suggested it could be unspecified or implementation-defined.
- Tom replied that it is more-or-less implied at present.
- PBrett agreed.
- Tom summarized the discussion; we agree that we want revised wording for this case but that we don't quite have what we want yet.
- Tom said he will inform LWG that we'll continue iterating on the wording with the intent to have something approved by our next meeting in two weeks.
- Hubert agreed with the summary.
- Victor gave a thumbs up.
- [P2558R1: Add @, \\$, and ` to the basic character set](#)
 - *[Editor's note: D2558R1 was the active paper under discussion at the telecon. The agenda and links used here reference P2558R1 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - Steve reported that the prose was updated to record the results of prior discussions in order to better explain the intent; the wording has not been changed.
 - Steve presented the paper and noted that section 3 is new.
 - Tom suggested adding a comment in section 3.1 (Universal Character Name) to indicate the character corresponding to `\u0060`.
 - PBrett reported a typo in section 3.4, "sting literal" should be "string literal".
 - PBrett noted that, with respect to existing use of these characters, they are usually used for convenience where another mechanism could be used.
 - Steve agreed and noted that such use generally occurs in contexts that require some kind of magic and where they can generally be escaped in some way.
 - Tom stated that, with regard to raw string literals, a reason to exclude the new characters in the delimiter portion is because these characters might acquire meaning in the future that could become problematic.
 - Tom expressed a preference to exclude ``` for now so that we can preserve it for use as a new type of string literal.
 - *[Editor's note: Such exclusion is unnecessary; the raw string literal delimiter pattern is bounded by a double quote and a parenthesis. Allowing use of ` in between those poses no ambiguity for a hypothetical new string literal delimited by `.]*
 - Tom suggested the paper explicitly note that the proposed changes enable these characters to portably be used in character literals by virtue of being encoded as a single code unit.
 - Steve agreed to update the paper.
 - PBrett reported another typo in section 3.3, "invariant" should be "invariant".
 - Jens expressed a continuing interest in the paper showing examples of behavioral changes.

- Hubert noted that such examples should be added to annex C.
- Steve reported two known compatibility issues:
 - Use of a UCN to name one of these characters in stringification.
 - Use of a UCN to name one of these characters as an argument to a function-like macro that does not use the corresponding parameter.
- Steve stated that SG22 may want to review these updates.
- Steve suggested it may suffice to forward an updated paper via an SG16 mailing list review.
- Tom agreed.
- Tom stated that the next meeting will be May 11th and will hopefully include review of an updated revision of D2572R0 (std::format() fill character allowances).

May 11th, 2022

Draft agenda:

- [P2286R8: Formatting Ranges](#)
 - Review and approve final wording updates.
- [P2558R1: Add @, \\$, and ` to the basic character set](#)
 - Continue review pending the availability of an updated revision.

Attendees:

- Charles Barto
- Hubert Tong
- Jens Maurer
- Mark de Wever
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [P2286R8: Formatting Ranges](#)
 - *[Editor's note: D2286R8 was the active paper under discussion at the telecon. The agenda and links used here reference P2286R8 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - Victor summarized recent wording changes worked out on the SG16 mailing list.
 - Victor asked if "code point" should be preferred over "character" in the proposed wording for [format.string.escaped]p2.
 - Tom replied that he is unaware of any normative use of "code point" in the standard today.
 - Victor responded that it is used in the wording for format field width estimation.

- Hubert stated that the usage there is in a Unicode specific context and that "character" is probably most appropriate here.
- Hubert pointed out that, in [format.string.escaped]p2.3, it is odd that `c` is defined as a character, but then compared with UCS scalar values.
- Jens agreed and proposed substituting "character" for "UCS scalar value" in paragraph 2.3.1 and in the header of the associated table.
- Jens suggested doing likewise in paragraph 2.3.3.
- Hubert argued that a change is not needed in 2.3.3 due to the use of "corresponds".
- Tom noted the use in that paragraph is also a Unicode specific context.
- Tom asked Charles if he continues to have concerns regarding the lack of specification for determining the boundaries of ill-formed code unit sequences.
- Charles replied that he does and that he would like to see it addressed via a reference to the [WHATWG Encoding Standard](#).
- Tom responded with uncertainty whether such a normative reference is possible given the lack of versioning around that standard.
- Charles suggested that the method specified in the WHATWG standard could be replicated in the C++ standard; we want the "maximal subpart" behavior described by policy option 2 in [Unicode PR-121](#).
- Hubert asked if that policy is defined for all UTF encodings.
- Charles replied that it is.
- PBrett asked what the motivation is for rigorously specifying how the boundaries of ill-formed code unit sequences are determined.
- Charles replied that the goal is to ensure consistent output, but then noticed that, in this case, the method used does not appear to be observable since each code unit of the sequence is written to the output anyway.
- Tom agreed that it should not matter for self-synchronizing encodings.
- Charles noted that this will be the first instance of Unicode UCD properties being normatively required by the C++ standard.
- Charles suggested that, if we're ok with such normative use, we could revisit the wording for estimated format field widths to make the uses there normative as well.
- [Editor's note: [\[format.string.std\]p11](#) specifies normative encouragement of behavior that depends on UCD properties in order to identify extended grapheme cluster boundaries.]
- [Editor's note: This would not be the first normative use of the UCD properties; [\[lex.name\]p1](#) requires the `XID_Start` and `XID_Continue` properties to determine identifier boundaries and validity.]
- Jens requested that such changes not be handled via this paper.
- Steve asked how much data is required for the new uses of the `General_Category` and `Grapheme_Extend` properties.
- Victor replied that the necessary data fits in ~1K.
- Charles agreed and shared a [link to code](#) used to implement a grapheme break algorithm that uses the `Grapheme_Cluster_Break` and `Extended_Pictographic` properties.
- Charles noted that some creative packing is necessary to get the size that small.
- Tom expressed surprise that `Grapheme_Extend` is small.
- Victor replied that it is composed of a number of compressable ranges.
- [Editor's note: The set of all characters that satisfy the `Grapheme_Extend=yes` property can be viewed [here](#); that set comprises 2090 code points in Unicode 14.]

- **Poll: Forward D2286R8 to LWG with 2.3.1 and associated table revised to substitute "character" for "UCS scalar value" as discussed for inclusion in C++23**
 - **Attendance: 8**
 - **No objection to unanimous consent.**
- [P2558R1: Add @, \\$, and ` to the basic character set](#)
 - *[Editor's note: D2558R1 was the active paper under discussion at the telecon. The agenda and links used here reference P2558R1 since the links to the draft paper were ephemeral. The published document may differ from the reviewed draft revision.]*
 - Steve introduced the changes made since the last review; just the addition of annex C wording.
 - **Poll: Forward D2558R1 to EWG for inclusion in C++26**
 - **Attendance: 8 (1 abstention)**
 - **SF F N A SA**
3 1 3 0 0
 - **Consensus in favor.**
 - Steve stated that he would follow up with SG22 with regard to issues found that were not discussed in WG14.
 - *[Editor's note: Steve did so via a [post to the C liaison list](#).]*
- Tom stated that the next meeting is scheduled for May 25th.

May 25th, 2022

Draft agenda:

- [D2572R0: std::format\(\) fill character allowances](#)
 - Continue review pending the availability of an updated revision.
- [L2/22-072R: Proposal for amendments to UAX#9 and UAX#31](#)
 - Review for familiarity and relevance to [P1949: C++ Identifier Syntax using Unicode Standard Annex 31](#).

Attendees:

- Charles Barto
- Hubert Tong
- Jens Maurer
- Mark de Wever
- Robin Leroy
- Steve Downey
- Tom Honermann

Meeting summary:

- In honor of a new attendee, a round of introductions was conducted.

- [D2572R0: std::format\(\) fill character allowances](#)
 - Tom presented the paper.
 - Robin pointed out a spelling error; "IDIOGRAPHIC" -> "IDEOGRAPHIC" (two occurrences).
 - Charlie explained that the ABI mitigation technique discussed in the "Future considerations and ABI" section relies on persistence of at least the fill character portion of the format string but such persistence is not otherwise currently required because the format string is evaluated at compile-time.
 - Charlie stated he could imagine ways of accomplishing the goal though.
 - Tom asked for confirmation that the Microsoft implementation is already shipping and locked into its current ABI.
 - Charlie confirmed that is the case.
 - Tom stated that he would add a note to the ABI section stating that some implementations are already locked in to their current behavior.
 - Steve commented that there are escape hatches and that other extension means are possible should the need arise.
 - Charlie explained why implementing ABI resiliency would likely impose dynamic memory management costs including possible lifetime management challenges.
 - Jens reported finding it a bit concerning that the estimated width of a character would be honored in some cases but not in others, but recognized the trade offs involved.
 - Jens stated that boilerplate wording is needed within the format section in order for the proposed use of "U+007B LEFT CURLY BRACKET" and "U+007D RIGHT CURLY BRACKET" to be applicable to the literal encoding.
 - Tom stated that the proposed wording changes to table 64 need work; in "if that value is negative", it is not clear whether "value" refers to "*n*" or to "the width of the formatting argument".
 - Jens requested that "estimated" be inserted before "width" in "the width of the formatting argument".
 - Hubert stated that "formatting argument" doesn't sound like the right term in this context; it should probably be "formatted argument".
 - Tom reported that this term was used for consistency with wording elsewhere but that he would review and try to improve.
 - Jens requested that the note following table 64 be modified to replace "ignored" with "assumed to be 1".
 - Tom agreed.
- [L2/22-072R: Proposal for amendments to UAX#9 and UAX#31](#)
 - Tom provided a brief introduction.
 - Hubert asked if it is necessary to address the UAX31-R3 conformance concern for C++23.
 - Tom replied that he did not believe so since the annex is non-normative.
 - *[Editor's note: Zoom crashed for Tom and it took him several minutes to get reconnected. Jens assured him that the time missed primarily concerned the flogging of a dead horse.]*
 - Jens asked what version of Unicode is expected to receive the proposed amendments.
 - Robin replied that Unicode 15 is expected to have these updates and that more significant normative changes are anticipated for Unicode 16.
 - Robin stated that Unicode 15 is expected to be released in September.
 - Jens observed that September would be just in time for adoption in C++23.

- Steve suggested that the annex could be updated to claim non-conformance with UAX31-R3.
- Hubert agreed and noted that we may not want to change our dated UAX31 reference at that late point in the C++23 release cycle.
- Jens proposed that we proceed with an NB comment on the C++23 committee draft to request upgrading the bibliography reference for UAX31 to Unicode 15.
- Jens explained that, since the new Unicode release won't be available before then, it won't be possible to act on a core issue and an NB comment would end up being required anyway.
- Robin directed discussion to allowances for U+200E LEFT-TO-RIGHT MARK (LRM) and U+200F RIGHT-TO-LEFT MARK (RLM) to be used in combination with other whitespace.
- Robin stated that example wording can be found in the Ada 2012 reference manual; [section 2.2 paragraph 7 1/3, "Lexical Elements, Separators, and Delimiters"](#) states:

One or more other_format characters are allowed anywhere that a separator is; any such characters have no effect on the meaning of an Ada program.
- Jens noted that this would be a new kind of whitespace for C++ since sequences of these marks by themselves would not constitute whitespace.
- Jens expressed curiosity regarding "implicit directional marks" as discussed in L2/22-072R.
- Robin replied that "implicit directional marks" is discussed in [UAX #9 section 2.6, "Implicit Directional Marks"](#).
- Robin explained that, per [UAX #9 section 6.5, "Conversion to Plain Text"](#), such marks may be implicitly inserted during conversion to plain text for text subject to protocol [UAX9-HL4](#) and that Unicode 15 will recommend that protocol for source code text.
- Hubert asked if it would make sense to prohibit sequences consisting of more than one of these marks.
- Robin replied that he knew of no motivation for doing so; that the presence of multiple marks should not pose any negative consequences.
- Jens expressed opposition to these marks constituting whitespace separation in isolation.
- Tom agreed.
- Jens noted that specification of LRM and RLM in whitespace will have to target C++26 as C++23 is now closed to new language changes.
- Jens suggested that such a change could be adopted as a DR against C++23 to encourage recognition of these marks as a conforming extension in prior language modes.
- Jens stated that a paper will be needed and that it should await the availability of Unicode 15.
- Tom stated he would file a SG16 github issue to track the request for such a paper.
- [*Editor's note: Tom filed [SG16 issue 74: Extend whitespace to include NEL, LS, PS, LRM, RLM, and maybe ALM.](#)*]
- Tom noted that this isn't a particularly urgent issue to address.
- Jens countered that it would be helpful to prevent obfuscated display of source code and that the desire to avoid such confusion has gained prominence in recent times.
- Jens directed discussion towards future conformance with UAX31-R3 and that there are questions about `Pattern_White_Space` that need to be answered.

- Robin asked which `Pattern_White_Space` characters are not considered whitespace in C++.
- Hubert listed them; they are all the ones outside the ASCII subset.
 - U+0085 NEXT LINE
 - U+200E LEFT-TO-RIGHT MARK
 - U+200F RIGHT-TO-LEFT MARK
 - U+2028 LINE SEPARATOR
 - U+2029 PARAGRAPH SEPARATOR
- Hubert noted that the above characters can only appear in comments and character or string literals currently.
- Jens asked if it is the intent of UAX31-R3 to require that all of the characters in `Pattern_White_Space` be supported as whitespace.
- Robin replied that allowing a subset would render the requirement vacuous.
- Jens suggested the possibility of updating UAX31-R3 to specify a minimal subset.
- Robin responded that a sub-requirement like UAX31-R3a could be introduced; such sub-requirements can be found elsewhere in UAX #31.
- Steve noted that the current normative text of UAX31-R3 allows deviation by specifying a profile.
- Hubert asked what motivation exists for not accepting the other whitespace characters.
- Steve noted existing practice and suggested this be addressed in the future paper.
- Jens directed discussion to conformance with the `Pattern_Syntax` requirement of UAX31-R3.
- Robin expressed a belief that C++ conforms to that.
- Tom expressed curiosity with regard to the presence of `.` in `Pattern_Syntax` and its use within floating point literals.
- *[Editor's note: Tom's concern stems from the following note in the description of UAX31-R3. Is the use of `.` in floating point literals considered syntax or part of a literal?*

Note: When meeting this requirement, all characters except those that have the `Pattern_White_Space` or `Pattern_Syntax` properties are available for use as identifiers or literals.

]

- Hubert stated that this kind of confusion is why he is hesitant to declare conformance to UAX31-R3 prior to improved wording that will hopefully appear in Unicode 16.
- Jens summarized the three tasks identified so far:
 - For C++23, file an NB comment after the July plenary to update [\[uaxid.pattern\]](#) in annex E to state that conformance with UAX31-R3 is not claimed. At the same time, update the UAX references in the bibliography to refer to Unicode 15.
 - For C++26, author a paper to add LRM, RLM, and other `Pattern_White_Space` characters to the set of whitespace characters. If support for U+061C ARABIC LETTER MARK is also desired, that will require a profile to conform with UAX31-R3.
 - For C++26, update [\[uaxid.pattern\]](#) in annex E to claim conformance with UAX31-R3. At the same time, update the UAX references in the bibliography to refer to Unicode 16 (or later).
- Robin noted that Unicode 15 is planned for release on September 13th per <https://www.unicode.org/versions/beta-15.0.0.html>.

- Tom recalled Hubert mentioning on the mailing list that U+000D CARRIAGE RETURN (CR) can now be added to the basic character set.
- Hubert acknowledged and opined that we can do so as part of [P2348: Whitespaces Wording Revamp](#).
- Tom stated that CR presumably should have already been present because of the existence of the `\r` escape sequence.
- Jens explained that `\r` creates a requirement for literal encodings but not for the basic character set nor an allowance for its use in whitespace.
- Tom noted that a paper will be needed that targets SG15 and discusses the concerns and options available to implementations with regard to UAX9-HL4 and presentation of source code that contains right-to-left characters.
- [*Editor's note: Tom filed [SG16 issue 75: SG15 proposal for implementations that present source code to conform with UAX9-HL4](#) to track producing such a paper.*]
- Tom stated that the next meeting will be in two weeks, on 2022-06-08.

June 8th, 2022

Draft agenda:

- [D2572R0: std::format\(\) fill character allowances](#)
 - Final review pending the availability of an updated revision.
- Discuss survey questions to suggest for the 2023 C++ Developer Survey
 - Review questioned and topics suggested in the [mailing list discussion](#).

Attendees:

- Charlie Barto
- Hubert Tong
- Inbal Levi
- Jens Maurer
- Mark de Wever
- Peter Brett
- Steve Downey
- Tom Honermann
- Victor Zverovich

Meeting summary:

- [D2572R0: std::format\(\) fill character allowances](#)
 - PBrett lamented the lack of a published revision with a "P" designation and change history that reflects the evolution of the design and wording.
 - Tom stated, in response to preferences expressed by Victor on the mailing list, that he will add a drafting note regarding the change of "specifier" to "option" in the description of the

align grammar production so that LWG will be sure to review.

- Jens requested that the drafting note regarding note renumbering be removed since the LaTeX machinery will handle that automatically.
- Tom stated that he would add a note following the format examples that mentions that the clown face emoji has an estimated length of two.
- PBrett suggested adding an example with a formatting argument that contains a character with an estimated width other than one; essentially an example that swaps the fill character and formatting argument in example `s7`.
- Jens requested that note X be modified to drop "estimated" and to replace "width of the fill character" with "width of **any** fill character".
- Jens expressed distaste for the existing wording in [\[format.string.std\]p11](#) that states "estimated width of ... UCS scalar values"; UCS scalar values are not characters.
- Hubert noted a missing "the" in the same paragraph; "the sum of **the** estimated widths".
- **Poll 1: Revise D2572R0 "std::format() fill character allowances" as discussed, and forward the paper so revised to LEWG as the recommended resolution of LWG3576 and LWG3639.**
 - **Attendance: 8 (2 abstention)**
 - **S F F N A S A**
3 3 0 0 0 0
 - **Consensus in favor.**
- Discuss survey questions to suggest for the 2023 C++ Developer Survey
 - PBrett proposed separating the survey questions into two categories:
 - The form of the source code; how the source code is written.
 - The facilities used to perform text processing.
 - PBrett suggested that questions about tools might comprise an additional category.
 - Inbal suggested asking for input on what topics most urgently require solutions in the standard.
 - Tom replied that a free form question could be used for that and that the current developer survey presents such free form responses as a word cloud.
 - PBrett asserted that the questions should address the topics we most want to learn about.
 - PBrett suggested asking what facilities programmers are using in place of standard facilities like `std::regex` and `std::locale` that are known to have significant design issues.
 - Hubert noted that the standard notion of locale encompasses both interface and locale identification; a programmer may use `std::locale` or the C locale facilities for locale identification, but then use alternate facilities for locale dependent behavior.
 - PBrett pondered whether the facilities programmers actively use to support internationalization and localization is one of the topics we are most ignorant of.
 - Hubert responded that there is speculation that programmers avoid the standard facilities but that we don't have data to confirm that.
 - Steve stated that Bloomberg actively avoids the standard locale related facilities.
 - Victor suggested it might be helpful to ask if programmers are intentionally using the standard locale facilities and noted that many do so inadvertently.
 - Victor noted that the questions need to consider C and C++ locale facilities as distinct facilities.
 - PBrett suggested structuring the questions as:

- "Do you provide internationalization support", and
 - "If so, how do you provide internationalization support".
- Victor stated that those questions should include multiple selection responses that include C, C++, POSIX, etc...
- PBrett suggested adding ICU and other popular packages.
- PBrett asked for additional topics that we might be particularly ignorant about.
- Steve suggested asking if programmers are still having to work with multiple character encodings within their main application and, if so:
 - whether they transcode at application boundaries and work exclusively with Unicode internally, or
 - whether they work directly with data in whatever character encoding it is provided in.
- PBrett suggested it would be useful to know how often programmers use regular expressions where the pattern is not known until run-time.
- Victor pondered whether Hana coerced Peter to ask that question.
- PBrett responded that she did not, but that the question does concern whether and to what extent CTRE is a suitable substitute for `std::regex`.
- Steve agreed that it would be useful to understand what the requirements for a replacement are.
- Charlie stated that a replacement would need to be more ABI resilient but that there is no need to pass compiled regular expression objects across module boundaries.
- Steve suggested asking which regular expression languages programmers are using.
- PBrett responded that question requires an "I don't know" response option.
- Tom asked if it would be helpful to know how many programmers are using `TCHAR` in Windows environments.
- Charlie reported suspicion that many programmers are still using that.
- Tom wondered what we might do with such data if we had it.
- PBrett suggested it would be interesting to know what libraries programmers are using for Unicode algorithm support and string classes.
- Tom asserted that that question would need to be multiple choice with an "other" option.
- Tom recalled one of the questions Peter had suggested on the mailing list, "what language(s) do you use in identifiers and comments?", and asked whether the question was intended to probe the languages used or whether characters outside the basic character set are being used.
- PBrett replied that the interest is in the language; the goal is to find out which scripts are being used.
- Tom recalled one of the questions Zach suggested, "How often do you use multiple Unicode normalization forms in the same program?" and commented that this is similar to the encoding question; whether programmers normalize at program boundaries or not normalize at all.
- Steve stated this is an important consideration for deciding if normalization belongs in the type system.
- Tom mentioned that Zach also posed questions about collation support.
- Steve replied that collation needs depend on context; data in a database is likely to be ordered in a locale independent manner but may need to be reordered for presentation in a user's locale.
- Inbal asked if serialization is within the purview of SG16.

- Tom replied that it could be for producing and consuming text formats.
- Inbal stated that, given a list of keywords, it would be possible to scrape stackoverflow.com for related questions.
- Tom wondered about asking if programmers place locale constraints on their users; for example, whether they require use of UTF-8.
- Hubert responded that such a question won't garner a 100% yes answer, so may not be so helpful.
- Tom replied that it might be useful to help guide where we invest effort; if lots of products support non-UTF-8 environments, then we know to focus more broadly.
- Hubert agreed with that perspective.
- PBrett stated that a transcoding facility remains high on our priority list and asked if the question about internal encodings and translation at program boundaries was intended to probe actual need.
- Tom responded that he had not thought about that relationship concretely.
- Steve suggested that question is more related to how many programmers need to operate directly on text in multiple encodings and if facilities are needed to do so.
- Hubert stated that gathering interest in a rope class would be useful.
- Tom asked if that would be for the case of stringing together blocks of text that are differently encoded.
- Hubert responded affirmatively.
- PBrett noted such a type is also useful in cases where buffers are in different places.
- Tom pondered what we would do with data regarding which character types are in use; for example, whether we would choose not to focus on `char32_t` as anything other than a code point type.
- Steve suggested asking if programmers use `signed char` and `unsigned char` for text as opposed to for use as small integers like `int8_t` and `uint8_t` or for other forms of bit manipulation.
- Hubert responded that `unsigned char` is likely used for UTF-8.
- Victor responded that `unsigned char` is often used for `uint8_t` and that this causes formatting confusion.
- Inbal asked if C and C++ compatibility is important.
- Tom replied that it is helpful to decide if we need low level C utilities that are exposed via C++ wrappers.
- Tom added that JeanHeyd has been pursuing the approach of getting low level facilities for transcoding through WG14 before continuing work on transcoding support in WG21.
- PBrett raised the question of which kind of C string interfaces are important; null terminated vs Pascal strings.
- PBrett mentioned WG14 Annex K and noted that Microsoft can't ship it due to conflicts with their historical secure function implementation.
- Hubert returned to the topic of localization and stated it would be useful to know if programmers customize locale formatting or just rely on default formatting.
- Tom stated that he will draft a Google doc with an initial set of questions based on this discussion that we can all comment on and contribute to.
- Further discussion ensued regarding stability vs the need to evolve and fix defects.
- Tom stated that the next meeting will be on 2022-06-22 and that, if we make good progress refining and suggesting survey questions in the interim, then we'll probably continue this

discussion then.